第 6 章練習　CPU 排程

> 本檔只含題目。請先自行建立狀態或變數表，再作答；解答在獨立檔案。

OS06-E01　非搶占排程計算

三個工作在 t=0 抵達，burst time 為 P1=8、P2=4、P3=2。

1. 分別計算 FCFS(P1,P2,P3) 與非搶占 SJF 的完成、等待與周轉時間。
2. 比較平均等待時間。
3. SJF 的結論需要什麼資訊假設？

OS06-E02　Round Robin

P1/P2/P3 均在 t=0 抵達，burst 為 5/2/3，time quantum q=2，context switch 成本暫忽略。

1. 畫出執行時間線。
2. 計算每個工作的完成、等待與回應時間。
3. 若 q=1，會改變哪些量？

OS06-E03　優先權與 starvation

高優先權 interactive tasks 持續到達，低優先權 batch job 等待時間無上限。

1. 何時可確定 starvation？
2. aging 如何避免？
3. 採用 fixed priority 且要求 bounded waiting 時，還需什麼？

OS06-E04　HRTICK CFS 逐事件推演

單 CPU Linux 3.2 CFS，target latency=6 ms、minimum granularity=0.75 ms、wakeup granularity=1 ms、GENTLE_FAIR_SLEEPERS=true、CONFIG_SCHED_HRTICK=y，切換成本為 0。P1/P2 在 t=0 runnable，P3 睡眠至 t=9；burst 為 8/4/2 ms，三者 nice=0、weight=1024。P1、P2、P3 的初始 vruntime 都是 0，P1/P2 同值時 P1 先入 tree。

1. 計算 runnable P1/P2 時的 period、ideal_runtime 與每 1 ms 實體時間增加的 vruntime。
2. 列 t=0、3、6、9、11、12、14 的 current、其他 ready、min_vruntime 與切換原因。
3. 由事件推導完整 Gantt，不得在 t=8 插入沒有事件的切換。
4. P3 在 t=9 的 sleep placement 為何？它會在 6–9 的哪個時間點搶占 P1？

第 6 章練習　CPU 排程 （續）

OS06-E05　週期 tick 的嚴格大於

沿用 OS06-E04 的 task 與初始狀態，但關閉 CONFIG_SCHED_HRTICK，假設一般 tick 每 1 ms 在整數時刻執行。

1. t=3 與 t=4 的 delta_exec 分別是多少？哪個時刻符合 `check_preempt_tick` 第一條件？
2. 推導新的 Gantt 與 P1/P2/P3 completion。
3. 為什麼不能把「ideal_runtime=3 ms」解讀成每個 task 一定在 t=3、6、9 精確切換？
4. 若 P3 在 t=8.5 醒來且重新啟用 HRTICK，舊的 t=9 Gantt 哪個區間先失效？

OS06-E06　CPU 公平與同步語意分離

兩個 threads 共用一個 CFS runqueue。A 的 weight=1024，B 的 weight=335，period=6 ms。B 先把資料寫入 buffer，再以 ordinary non-atomic store 設 ready=1；A 輪到時以 ordinary load 檢查 ready。

1. 計算兩者的 ideal_runtime 與相對 CPU share。
2. 較高 weight 是否能取代 A/B 之間的 mutual exclusion？用兩個 writers 反例說明。
3. 什麼 synchronization contract 才能讓 A 在 ready=1 後安全看到 buffer？
4. 若以 mutex 保護 counter++，mutex 提供哪個性質？它是否自動是 release/acquire publication protocol？
5. 分別說明 memory barrier、atomic operation、mutex 與 semaphore 對互斥及可見性各提供什麼，不能只列名稱。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 6 章練習　CPU 排程 （續）

OS06-E07　RM 充分界、逐事件 EDF 與短期限

> 單 CPU 可立即搶占；兩 task 同步於 0 起釋放，沒有 blocking、自我暫停、I/O 或額外成本，C 是每筆實際且最壞的 CPU 需求。甲組 $\tau_1=(C=2,T=D=5)$、$\tau_2=(C=4,T=D=8)$；同 task 的舊 job 先做，EDF 平手保留 current，否則依 task/job 編號。完成恰在期限算成功，超時 job 不丟棄。
>
> 1. 算甲組 U 與兩 task 的 RM 充分界；可否只因此判定 RM 成功或失敗？迭代 $w^{(0)}=C_2$，$w^{(k+1)}=C_2+\lceil w^{(k)}/T_1\rceil C_1$。
> 2. 列 RM 在 [0,10) 的每段執行、job 完成及期限，核對前題固定點。
> 3. 改用 EDF，列 [0,16) 每次 release/completion 後的選擇、剩餘需求與絕對期限，特別解釋 t=5、8、10、15；核對所有在 16 或之前到期的 jobs。有限前綴通過後，哪個定理能保證全序列？
> 4. 乙組 $\tau_1=(1,2,2)$、$\tau_2=(2,4,4)$，畫一個共同週期的 RM 時間表，證明超過充分界不代表 RM 失敗。
> 5. 丙組兩 task 均 C=2,T=10,D=3。用長度3的區間需求證明任意政策都失敗；解釋甲組為何可用 U 判定理想 EDF，丙組卻不行。甲組若加入鎖阻塞，舊結論是否仍足夠？

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study